Storing Data, Step by Step

class 2 of 5 · 40 minutes · every operation, one at a time

Dinesh Rawat

Instructor · Shorter Loop × Snapied

> Yesterday you created a database and a users table by hand in Workbench, and wrote INSERT and SELECT yourself. Today you do the same work from JavaScript, one operation at a time. For each step you will see three things: the JavaScript you write, the SQL Sequelize turns it into, and the result. By the end you will have stored a real user and their resumes in MySQL from code, read them back, and changed one, and you will understand exactly what each line did to the database.

0 The setup before any step

Three model files describe the tables, and one connection points at your MySQL. You wrote versions of these already. Here they are together so every step below has its foundation.

config/database.js · the MySQL connection

```
// config/database.js
// The single connection to our MySQL database.
// Every other file talks to the database through this one object.

const { Sequelize } = require('sequelize');

const sequelize = new Sequelize(
  'resume_db',        // the database name (we create this in MySQL Workbench first)
  'root',             // your MySQL username
  'your_password',    // your MySQL password (set when you installed MySQL)
  {
    host: 'localhost', // MySQL runs on your own machine
    dialect: 'mysql',  // tells Sequelize to speak MySQL
    logging: false,    // set to console.log to see every SQL query it sends
  }
);

module.exports = sequelize;
```

models/user.js

```
// models/user.js
const { DataTypes } = require('sequelize');
const bcrypt = require('bcryptjs');
const sequelize = require('../config/database');

const User = sequelize.define('User', {
  name:     { type: DataTypes.STRING, allowNull: false },
  email:    { type: DataTypes.STRING, allowNull: false, unique: true,
              validate: { isEmail: true } },
  password: { type: DataTypes.STRING, allowNull: false },
});

// hash the password before every save
User.beforeCreate(async (user) => {
  const salt = await bcrypt.genSalt(10);
  user.password = await bcrypt.hash(user.password, salt);
});

User.prototype.checkPassword = function (plainText) {
  return bcrypt.compare(plainText, this.password);
};

// ---- THIS model's relationships live in THIS file ----
// It receives all the models, so it never has to require Resume itself.
// That is what avoids the circular require. Called once, after all
// models are loaded (see models/index.js).
User.associate = (models) => {
  User.hasMany(models.Resume, { foreignKey: 'userId', onDelete: 'CASCADE' });
};

module.exports = User;
```

The Resume model and the relationship:

models/resume.js

```
// models/resume.js
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Resume = sequelize.define('Resume', {
  title:   { type: DataTypes.STRING, allowNull: false },
  summary: { type: DataTypes.TEXT },
});

// ---- THIS model's relationship lives in THIS file ----
Resume.associate = (models) => {
  Resume.belongsTo(models.User, { foreignKey: 'userId' });
};

module.exports = Resume;
```

models/index.js

```
// models/index.js
// This file no longer defines relationships. Each model does that itself,
// in its own associate() function. This file just loads the models and
// then calls each associate() once everything is loaded.

const sequelize = require('../config/database');
const User = require('./user');
const Resume = require('./resume');

const models = { User, Resume };

// Now that all models are loaded, wire up each one's relationships.
Object.values(models).forEach((model) => {
  if (model.associate) model.associate(models);
});

module.exports = { sequelize, ...models };
```

Where the relationships live. Each model declares its own relationship inside an `associate` function in its own file: `User.hasMany` is in user.js, `Resume.belongsTo` is in resume.js. The `associate` function receives all the models as an argument, so a model never has to import the other one directly, which avoids a load-order trap. `index.js` is now just a small loader: it loads every model, then calls each `associate` once they are all ready.

Import from index.js in your code, never the single model files. The relationships are only wired up after index.js runs its loader. Import `user.js` directly and you get a User that does not know about resumes yet, so joins fail. Always import from index.

1 Step one: make sure the tables exist

What you write

```
const { sequelize, User, Resume } = require('./models');

await sequelize.sync();   // no force, so existing data is kept
```

What it does. `sync()` looks at your models and creates any table that does not exist yet. Without `force: true` it leaves existing tables and their data untouched, so it is safe to run again and again. On the very first run it creates `users` and `resumes`. After that it does nothing, which is exactly what you want.

> **Never use sync({ force: true }) on data you care about.** The `force` option drops every table first, then rebuilds them empty. Fine on the first day of learning, a disaster on real data. Plain `sync()` is the safe default.

2 Step two: store one user

What you write

```
const user = await User.create({
  name: 'Himanshu',
  email: 'himanshu@example.com',
  password: 'secret123',
});
console.log('Saved user #' + user.id);
```

The SQL Sequelize sends

```
INSERT INTO `Users`
  (`name`, `email`, `password`, `createdAt`, `updatedAt`)
VALUES ('Himanshu', 'himanshu@example.com', ..., NOW(), NOW());
```

What prints

```
Saved user #1
```

Read it line by line. You called `User.create` with a plain object. Sequelize built the `INSERT`, filled in `createdAt` and `updatedAt` itself, and let MySQL assign the `id`. It handed you back the saved user, so `user.id` is now `1`. You wrote no SQL. You described what you wanted and the ORM did the rest.

> **The returned object is the saved row.** After `create`, `user` is the real record, id and all. You will use `user.id` in the very next step to link resumes to this person. This is the everyday rhythm: create a thing, keep the returned object, use its id.

3 Step three: store two resumes for that user

What you write

```
await Resume.create({
  title: 'Full Stack Intern',
  summary: 'Built REST APIs with Node, Express and MySQL.',
  userId: user.id,
});

await Resume.create({
  title: 'QA Intern',
  summary: 'Manual test cases and API tests with Postman.',
  userId: user.id,
});
console.log('Saved 2 resumes');
```

The SQL for each

```
INSERT INTO `Resumes`
  (`title`, `summary`, `userId`, `createdAt`, `updatedAt`)
VALUES ('Full Stack Intern', '...', 1, NOW(), NOW());
```

What prints

```
Saved 2 resumes
```

The important part is `userId: user.id`. That single value is what links each resume to Himanshu. Both resumes carry `userId: 1`, so both belong to user 1. This is the foreign key from yesterday, now set from code. Forget this line and you create an orphan resume that belongs to nobody.

4 Step four: read this user's resumes back

What you write

```
const resumes = await Resume.findAll({ where: { userId: user.id } });
console.log('This user has', resumes.length, 'resumes:');
resumes.forEach(r => console.log('  -', r.title));
```

The SQL

```
SELECT id, title, summary, userId, createdAt, updatedAt
FROM `Resumes`
WHERE `userId` = 1;
```

What prints

```
This user has 2 resumes:
  - Full Stack Intern
  - QA Intern
```

What happened. `findAll` reads many rows. The `where` narrows it to rows where `userId` is 1, so you get exactly this user's resumes and no one else's. It returns an array, which is why you loop over it. `findAll` with no `where` would return every resume in the table.

Method	Returns	Use when
`findAll`	an array of rows	you want many: all resumes for a user
`findByPk`	one row, by its id	you know the exact id
`findOne`	the first matching row	you expect just one: a user by email

5 Step five: read one resume with its owner (a join)

What you write

```
const first = await Resume.findByPk(resumes[0].id, { include: User });
console.log('Resume "' + first.title + '" belongs to ' + first.User.name);
```

The SQL, a real JOIN

```
SELECT Resume.*, User.name, User.email
FROM `Resumes` AS `Resume`
LEFT OUTER JOIN `Users` AS `User`
  ON `Resume`.`userId` = `User`.`id`
WHERE `Resume`.`id` = 1;
```

What prints

```
Resume "Full Stack Intern" belongs to Himanshu
```

This is yesterday's join, from code. `include: User` told Sequelize to fetch the owner alongside the resume. It wrote a `LEFT OUTER JOIN` matching `userId` to the user's `id`, exactly the join you typed by hand in Workbench. The owner arrives attached as `first.User`, so `first.User.name` is right there in the same object. One query, two tables, no manual stitching.

> It is first.User with a capital U. Sequelize names the attached owner after the model, `User`, not `user`. Writing `first.user.name` gives undefined and confuses everyone the first time. Match the model name.

6 Step six: change a saved row

What you write

```
first.title = 'Senior Full Stack Intern';
await first.save();
console.log('Updated title:', first.title);
```

The SQL

```
UPDATE `Resumes`
SET `title` = 'Senior Full Stack Intern', `updatedAt` = NOW()
WHERE `id` = 1;
```

What prints

```
Updated title: Senior Full Stack Intern
```

How updating works. You change a field on the object you already loaded, then call `save()`. Sequelize sees which field changed and writes an `UPDATE` touching only that column, and it refreshes `updatedAt` for you. It knows which row to update from the object's `id`, so the `WHERE id = 1` is added automatically. You never risk updating every row the way a hand-written UPDATE without a WHERE would.

7 Step seven: prove it in Workbench

The whole point of today is that this is real, shared data. Your code wrote to the same MySQL that Workbench reads. Open Workbench and run:

```
SELECT * FROM resumes;
```

```
+----+--------------------------+---------+--------+
| id | title                    | summary | userId |
+----+--------------------------+---------+--------+
|  1 | Senior Full Stack Intern | ...     |      1 |
|  2 | QA Intern                | ...     |      1 |
+----+--------------------------+---------+--------+
```

There are the rows your JavaScript created, including the title you changed in step six. The app and Workbench are two windows onto one real database.

> This is the bridge, complete. Yesterday you were the one typing INSERT, SELECT and UPDATE in Workbench. Today your JavaScript did all of it, through the models, and Workbench shows the same rows. That is exactly what every backend does: turn a user's click into a saved row, and a page load into a query. You have now done both directions.

8 What the database refuses

The rules on your models are enforced on this real data. Try to break them from code and MySQL stops you, which is the protection you want:

Try this	What happens
User.create with an email that already exists	SequelizeUniqueConstraintError. the UNIQUE rule holds.
User.create with no name	a notNull violation. allowNull: false holds.
Resume.create with userId 999 (no such user)	a foreign key error. MySQL refuses an orphan resume.

None of these is JavaScript being difficult. Each is the database guarding the integrity of your data, catching a mistake before it becomes a broken row.

9 Before next class

Run all seven steps against the MySQL you set up in Class 1. Confirm each prints what you see above.

Open Workbench and run `SELECT * FROM resumes` after your code runs. Seeing your own rows, including the updated title, is the moment today is meant to give you.

Add a third resume from code, run step four again, and watch the count go from two to three.

Try one forbidden thing from the table in section 8 and read the error. Next class we make the password safe with bcrypt before it is ever stored.

Everything here ran live in class. If a part does not click, message me the file name and the line, and read it again tonight while it is fresh.